

The background of the image shows several industrial robotic arms and bases in a dark environment. The robots are illuminated with a cool blue light, highlighting their joints and structural elements. They are arranged in a way that suggests a factory or laboratory setting. The lighting creates strong reflections on the floor and emphasizes the metallic textures of the robots.

深圳市大族机器人有限公司

script脚本使用说明

——曾尉

1.基本接口

1.SetOverride(设置机器人运行速度)

value = 0.3 #这里的0.3指的是速度比即30%

cc_client.SetOverride(value)

注：由于安全原因，在运动过程中不允许修改机器人运动速度

2.SetPayload(设置机器人的动态负载)

load = 3 #这里指机器人末端法兰盘下所带负载的重量,单位kg

gravity_x = 0 #这里指机器人末端法兰盘下所带负载的重心,单位mm

gravity_y = 0

gravity_z = 0

cc_client.SetPayload(load,gravity_x,gravity_y,gravity_z)

3.ReadTCP(读取机器人TCP的坐标值)

```
tcp_name = 'TCP_Gripper'          #这里的TCP_Gripper指的是机器人需要读取的tcp名称  
cc_client.ReadTCP(tcp_name)
```

4.ReadUCS(读取机器人用户坐标系的坐标值)

```
ucs_name = 'Plane_user'           #这里的Plane_user指的是机器人需要读取的面坐标系名称  
#ucs_name = 'Line_user'           #这里的Line_user指的是机器人需要读取的线坐标系名称  
#ucs_name = 'Point_user'          #这里的Point_user指的是机器人需要读取的点坐标系名称  
cc_client.ReadUCS(ucs_name)
```

5.ConfigTCP(设置TCP的坐标值)

tcp_name = 'TCP_Gripper' #这里的TCP_Gripper指的是机器人需要设置的tcp名称

#这里设置的是TCP坐标值对应空间和姿态角x,y,z,rx,ry,rz,其中x,y,z单位为mm,rx,ry,rz的单位为度,下面的设置机器人用户坐标系的值同理

tcp_values = [0,0,0,0,0,0]

cc_client.ReadTCP(tcp_name,tcp_values)

6.ConfigUCS(设置用户坐标系的坐标值)

ucs_name = 'Plane_user'

ucs_values = [0,0,0,0,0,0]

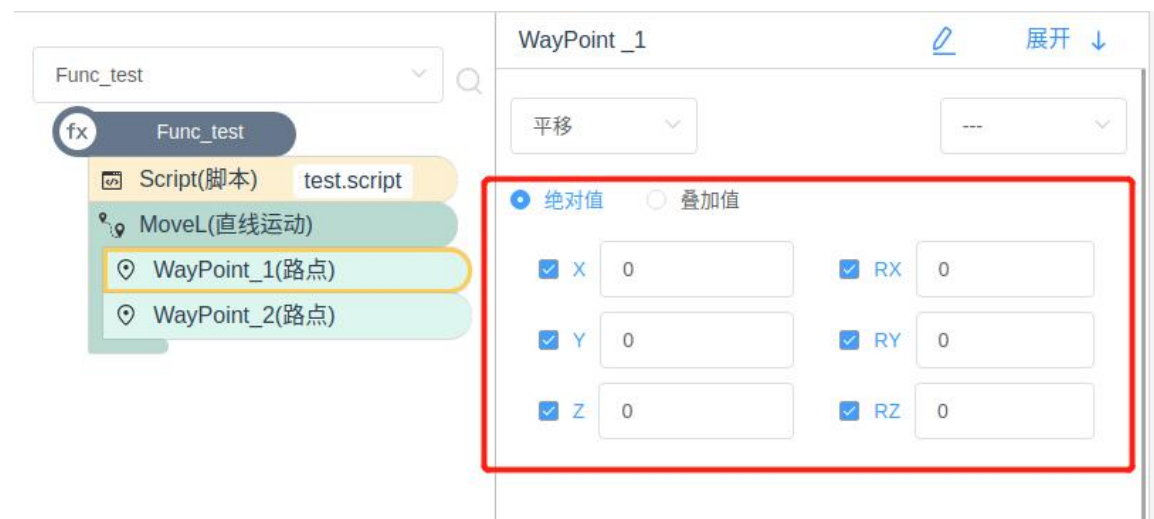
cc_client.ConfigUCS(ucs_name,ucs_values)

注：这里的设置用户坐标有个常用的场景，就是在加3D视觉的时候，可以用于设置动态TCP，这样就可使机器人沿着末端方向移动

例子：

```
pos = cc_client.ReadActPos()[6:12] #这里使用ReadActPos读取机器人当前的空间坐标
```

```
cc_client.ConfigUCS( 'Point_user' ,pos) #注意这里使用的是点坐标系
```



7.UcsTcp2Base(由指定用户坐标系和工具坐标系下的迪卡尔坐标位置计算基坐标系下的坐标位置)

```
user_pos = [0,-100,0,0,0,0]      #用户坐标系、tcp下的迪卡尔坐标
```

```
tcp_pos = [0,0,0,0,0,0]          #tcp的坐标值
```

```
ucs_pos = [420,100,445,180,0,180]  #用户坐标系的坐标值
```

```
read = cc_client.UcsTcp2Base(user_pos ,tcp_pos,ucs_pos ) #read->获得返回数值
```

8.Base2UcsTcp(由基坐标系下的坐标位置计算指定用户坐标系和工具坐标系下的迪卡尔坐标位置)

```
base_pos = [420,0,445,180,0,180]   #base下的迪卡尔坐标
```

```
tcp_pos = [0,0,0,0,0,0]            #tcp的坐标值
```

```
ucs_pos = [420,100,445,180,0,180]  #用户坐标系的坐标值
```

```
read = cc_client.Base2UcsTcp(base_pos ,tcp_pos,ucs_pos ) #read->获得返回数值
```

9.PCS2ACS(逆解, 由指定用户坐标系位置和工具坐标系下的迪卡尔坐标计算对应的关节坐标位置)

```
pos_PCS = [420,0,445,180,0,180]      #用户坐标系、tcp下的迪卡尔坐标
```

```
#参考关节坐标, 逆解出现多个解时需要根据参考关节坐标选取最终解
```

```
consult_ACS = [0,0,90,0,90,0]
```

```
tcp_pos = [0,0,0,0,0,0]              #tcp的坐标值
```

```
ucs_pos = [0,0,0,0,0,0]              #用户坐标系的坐标值
```

```
read = cc_client.PCS2ACS(pos_PCS,consult_ACS,tcp_pos,ucs_pos ) #read->获得返回  
数值(逆解返回的是J1-J6轴的关节角度)
```

10.readRobotState(读取当前机器人状态标志)

read_state = cc_clien.readRobotState() #read_state->获得返回数值(会返回数组)

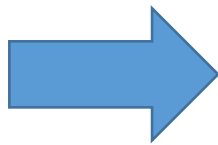
返回值	名称	数据类型	有效范围	内容
nMovingState	运动中状态	int	0/1	0: 机器人不处于运动状态 1: 机器人运动中
nEnableState	已使能状态	int	0/1	0: 机器人未使能 1: 机器人已使能
nErrorState	错误状态	int	0/1	0: 未发生错误 1: 有错误发生
nErrorCode	错误码	int	≥0	报错错误码
nErrorAxis	错误轴 ID	int	0~6	错误轴 ID
nBreaking	开抱闸状态	int	0/1	机器人轴处于开抱闸状态(受重力作用可能导致轴往下掉)
nPause	暂停状态	int	0/1	0: 未处于暂停状态 1: 已处于暂停状态
nEmergencyStop	急停状态	int	0/1	0: 未处于急停状态 1: 已处于急停状态
nSaftyGuard	安全光幕状态	int	0/1	0: 未处于安全光幕状态 1: 已处于安全光幕状态
nElectrify	已上电状态	int	0/1	0: 未上电状态 1: 已上电状态
nIsConnectToBox	电箱连接状态	int	0/1	0: 电箱未连接 1: 电箱已连接

nBlendingDone	路点运动状态	int	0/1	0: 路点运动中 1: 路点运动完成
nInPos	到位状态	int	0/1	0: 机器人实际位置还没有运动到命令位置 1: 运动到位(命令与实际位置基本没有误差)

11.ReadCurFSM(读取当前机器人状态机)

read_state = cc_clien.ReadCurFSM() #read_state->获得返回数值(会返回一个int型整数)

值	别名	含义
0	UnInitialize	未初始化
1	Initialize	初始化
2	ElectricBoxDisconnect	与电箱控制板断开
3	ElectricBoxConnecting	连接电箱控制板
4	EmergencyStopHandling	急停处理中
5	EmergencyStop	急停
6	Blackouting48V	正在切断本体供电
7	Blackout_48V	本体供电已切断
8	Electrifying48V	正在准备给本体供电
9	SaftyGuardErrorHandling	安全光幕错误处理中
10	SaftyGuardError	安全光幕错误
11	SafetyGuardHandling	安全光幕处理中
12	SaftyGuard	安全光幕
13	ControllerDisconnecting	正在反初始化控制器
14	ControllerDisconnect	控制器已处理于未初始化状态
15	ControllerConnecting	正在初始化控制器
16	ControllerVersionError	控制器版本过低错误
17	EtherCATError	EtherCAT 错误
18	ControllerChecking	控制器初始化后检查
19	Reseting	正在复位机器人
20	RobotOutofSafeSpace	机器人超出安全空间
21	RobotCollisionStop	机器人安全碰撞停车



22	Error	机器人错误
23	RobotEnabling	机器人使能中
24	Disable	机器人去使能
25	Moveing	机器人运动中
26	LonglogMoveing	机器人长点动运动中
27	RobotStopping	机器人停止运动中
28	RobotDisabling	机器人去使能中
29	RobotOpeningFreeDriver	机器人正在开启零力示教
30	RobotClosingFreeDriver	机器人正在关闭零力示教
31	FreeDriver	机器人处于零力示教
32	RobotHolding	机器人暂停
33	StandBy	机器人就绪
34	ScriptRunning	脚本运行中
35	ScriptHoldHandling	脚本暂停处理中
36	ScriptHolding	脚本暂停
37	ScriptStopping	脚本停止中
38	ScriptStopped	脚本已停止
39	HRAppDisconnected	HRApp 部件断开
40	HRAppError	HRApp 部件错误

12.stop(停止脚本，停止示教器页面正在执行脚本文件)

```
cc_client.stop()
```

13.ReadActPos(读取实际关节位置，空间位置，工具坐标，用户坐标)

```
read_pos = cc_client.ReadActPos()
```

```
ACS_pos = read_pos[0:6]      #读取关节坐标
```

```
PCS_pos = read_pos[6:12]    #读取空间坐标
```

```
TCP_pos = read_pos[12:18]   #读取tcp坐标
```

```
UCS_pos = read_pos[18:24]   #读取用户坐标
```

14. ReadPointByName(读取点位列表中点位的关节位置, 空间位置, 工具坐标, 用户坐标)

```
point_name = 'Point_1'      #点位列表中点位的名称  
read_pos = cc_client.ReadPointByName(point_name)  
ACS_pos = read_pos[0:6]     #读取关节坐标  
PCS_pos = read_pos[6:12]    #读取空间坐标  
TCP_pos = read_pos[12:18]   #读取tcp坐标  
UCS_pos = read_pos[18:24]   #读取用户坐标
```

注:HR.WV5.49.0.B27_CPS20220624.203软件版本以上的使用上面接口, 以下使用下面接口

```
point_name = 'Point_1'      #点位列表中点位的名称  
read_pos = cc_client.ReadPointList(point_name)
```

15. readDI(读取电箱通用数字输入状态)

`read_DI = cc_client.readDI(5)` `#read_DI`->获得返回数值,读取DI5的数值,高电平是1,低电平是0

16.readDO(读取电箱通用数字输出状态)

`read_DO = cc_client.readDO(5)` `#read_DO`->获得返回数值,读取DO5的数值,高电平是1, 低电平是0

17.readCI(读取电箱控制数字输入状态)

`read_CI = cc_client.readCI(5)` `#read_CI`->获得返回数值,读取CI5的数值,高电平是1,低电平是0

18.readCO(读取电箱控制数字输出状态)

`read_CO = cc_client.readCO(5)` `#read_CO`->获得返回数值,读取CO5的数值,高电平是1, 低电平是0

19. setDO(设置电箱通用数字输出状态)

cc_client.setDO(5,1) #设置DO5输出高电平,高电平是1, 低电平是0

20. setCO(设置电箱控制数字输出状态)

cc_client.setCO(5,1) #设置CO5输出高电平,高电平是1, 低电平是0

21.readBoxAI(读取电箱模拟量输入值)

read_AI = cc_client.readBoxAI(0) #read_AI->获得返回数值,读取AI0的数值

22.readAO(读取电箱模拟量输出值)

read_AO = cc_client.readAO(0) #read_AO->获得返回数值,读取AO0的数值

23.SetBoxAO(设置电箱模拟量输出值)

cc_client.SetBoxAO(0,3,1) #0代表设置AO0 3代表输出值 1代表选择模式是电压(2是电流)

24. readEI(读取末端通用数字输入状态)

```
read_EI = cc_client.readEI(0)    #read_EI->获得返回数值,读取末端DI0的数值,高电平是1, 低电平是0
```

25. readEO(读取末端通用数字输出状态)

```
read_EO = cc_client.readEO(5,1)  #read_EO->获得返回数值,读取末端DO0的数值,高电平是1, 低电平是0
```

26.setEO(设置末端通用数字输出状态)

```
cc_client.setEO(0,1)    #设置末端DO0输出高电平,高电平是1, 低电平是0
```

27.readEAI(读取末端模拟量输入值)

```
read_EAI = cc_client.readEAI(0)  #read_EAI->获得返回数值,读取末端AI0的数值
```

28. sendAndRecv(通信接口调用)

```
charsend = 'PauseScript'
```

```
cc_client.sendAndRecv(charsend)
```

#通过此接口可以调用HansRobotV5 通信协议接口里的大部分接口

29. sendVarValue(可以将script内的局部变量输出给程序的全局变量并使用)

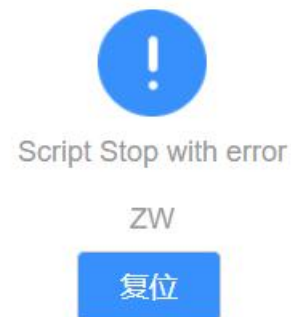
```
a = 'ZW'
```

```
cc_client.sendVarValue( 'a' ,a)
```

30.sendScriptError(弹窗报错)

```
cc_client.sendScriptError('ZW')
```

```
control.exit()
```



31. sendHRLog(打印日志)

charsend = 'PauseScript'

cc_client.sendHRLog(1,charsend) #1代表打印日志类型，警告为1，报警为2

2.运动接口

1.WayPoint和WayPoint2和WayPointEx(路点运动函数)

#type "MoveJ" 是0 "MoveL"是 1 "MoveC" 是2

move_type = 1

pcs_point = [420,0,445,180,0,180] #空间坐标

acs_point = [0,0,90,0,90,0] #关节坐标

TCP_name = 'TCP' #工具坐标系名

ucs_name = 'Base' #用户坐标系名

TCP_point = [0,0,0,0,0,0] #工具坐标系坐标值

UCS_point = [0,0,0,0,0,0] #用户坐标系坐标值

velocity = 500 #速度 线速度1000 角速度100

acc = 2500 #加速度 线加速度2500 角加速度360

radius = 0 #交融半径

IsJoint = 0 #是否使用关节角度作为目标点，如果move_type=0 时，则IsJoint 有效：
0：不使用关节角度 1：使用关节角度 使用的关节角度是空间坐标逆解得出的

nlSeek = 0 #是否使用探寻功能

nIOBit = 0 #使用探寻功能触发的DI是那个

nIOState = 0 #检测的 DI 状态，是高电平触发还是低电平触发

#如果 nlSeek 为 1，则开启检测 DI 停止，路点运动过程中如果电箱的 nIOBit 位索引的 DI
的状态=nIOState 时，机器人停止运动，否则运动到目标点完成运动

CmdID = 0 #当前路点 ID，可以自定义

strPointGuid = '0'

move_type=1: 目标空间位 move_type=2: 做为圆弧的目标位置 move_type=0无效

EndPos = [470,50,445,180,0,180]

#nMoveType=2: 做为圆弧的经过位置 move_type=0无效 move_type=1无效

AuxPos = [470,50,445,180,0,180]

#nMoveType=0 并 nIsUseJoint=1: 使用此关节坐标作为目标关节坐标 nMoveType=0 并

nIsUseJoint=0: 此关节坐标仅作为计算逆解时选解的参考关节坐标 nMoveType=1: 无效

nMoveType=2: 无效

PcsPos = [0, 0, 90, 0, 90, 0]

cc_client.WayPoint(move_type,pcs_point,acs_point,TCP_name,ucs_name,velocity,acc,
radius,isJoint, nIsSeek, nIOBit, nIOState, CmdID)

cc_client.WayPointEx(move_type,pcs_point,acs_point,TCP_point,UCS_point,velocity,a
cc,radius,isJoint, nIsSeek, nIOBit, nIOState, CmdID)

```
cc_client.WayPoint2(move_type,EndPos,AuxPos,PcsPos,TCP_name,ucs_name,velocity,  
acc,radius,isJoint, nlsSeek, nIOBit, nIOState, strPointGuid)
```

```
control.waitBlendingDone(cc_client) #等待路点运动完成
```

注：如果想要让运动连续不停顿，需要在到达目标点前下发下一个点位，最后等待路点运动完成即可

3.通信接口

1.modbus接口(机器人做从站)

ReadCoils(读取线圈 类型 可读可写)

addr = 0 #起始地址位

num = 3 #读取数量

read_coil = cc_client.MBSlave_ReadCoils(addr,num) #返回值read_coil为数组格式

ReadInputCoils(读取线圈 类型 可读不可写)

addr = 0 #起始地址位

num = 3 #读取数量

read_coil = cc_client.MBSlave_ReadInputCoils(addr,num) #返回值read_coil为数组格式

ReadHoldingRegisters(读取寄存器 类型 可读可写)

addr = 0 #起始地址位

num = 3 #读取数量

read_Register = cc_client.MBSlave_ReadHoldingRegisters(addr,num) #返回值

read_Register为数组格式

ReadInputRegisters(读取寄存器 类型 可读不可写)

addr = 0 #起始地址位

num = 3 #读取数量

read_Register = cc_client.MBSlave_ReadInputRegisters(addr,num) #返回值

read_Register为数组格式

ReadHoldingRegisters_Float(读取寄存器 类型 可读可写 数据 浮点数)

addr = 0 #起始地址位

num = 3 #读取数量

read_Register = cc_client.MBSlave_ReadHoldingRegisters_Float(addr,num) #返回值
read_Register为数组格式

ReadHoldingRegisters_Int(读取寄存器 类型 可读可写 数据 整数)

addr = 0 #起始地址位

num = 3 #读取数量

read_Register = cc_client.MBSlave_ReadHoldingRegisters_Int(addr,num) #返回值
read_Register为数组格式

ReadInputRegisters_Float(读取寄存器 类型 可读不可写 数据 浮点数)

addr = 0 #起始地址位

num = 3 #读取数量

read_Register = cc_client.MBSlave_ReadInputRegisters_Float(addr,num) #返回值

read_Register为数组格式

ReadInputRegisters_Int(读取寄存器 类型 可读不可写 数据 整数)

addr = 0 #起始地址位

num = 3 #读取数量

read_Register = cc_client.MBSlave_ReadInputRegisters_Int(addr,num) #返回值

read_Register为数组格式

WriteCoils(写多个线圈 类型 可读可写)

addr = 0 #起始地址位

vals = [1,1,0,0] #根据输出个数将数值按顺序成数组

cc_client.MBSlave_WriteCoils(addr,vals)

WriteCoil(写单个线圈 类型 可读可写)

addr = 0 #起始地址位

val = 1 #输出数据

cc_client.MBSlave_WriteCoils(addr,val)

WriteHoldingRegisters(写多个寄存器 类型 可读可写)

addr = 0 #起始地址位

vals = [2,3,1,5] #根据输出个数组将数值按顺序成数组

cc_client.MBSlave_WriteHoldingRegisters(addr,vals)

WriteHoldingRegister(写单个寄存器 类型 可读可写)

addr = 0 #起始地址位

val = 12 #输出数据

cc_client.MBSlave_WriteCoils(addr,val)

WriteHoldingRegisters_Float(写多个寄存器 类型 可读可写 数据 浮点数)

addr = 0 #起始地址位

vals = [2.2,3.1,1.0,5.4] #根据输出个数将数值按顺序成数组

cc_client.MBSlave_WriteHoldingRegisters_Float(addr,vals)

WriteHoldingRegisters_Int(写多个寄存器 类型 可读可写 数据 整数)

addr = 0 #起始地址位

val = 12.0 #输出数据

cc_client.MBSlave_WriteCoils(addr,val)

2.modbus接口(机器人做主站)

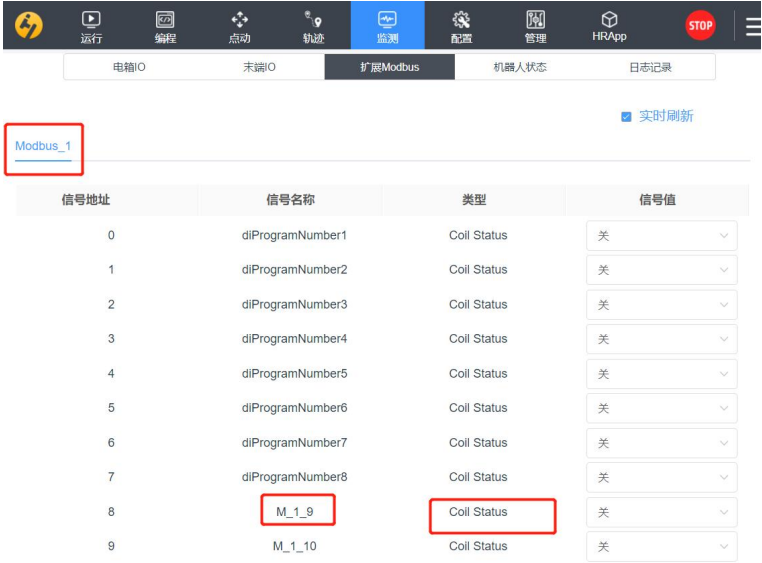
setModbus(写modbus地址位)

deviceName = 'Modbus_1' #modbus设备号

varName = 'M_1_9' #地址名称

val = 1 #输出数据,这里取决于数据类型,线圈输出就只能写0和1

cc_client.setModbus(deviceName,varName,val)



信号地址	信号名称	类型	信号值
0	diProgramNumber1	Coil Status	关
1	diProgramNumber2	Coil Status	关
2	diProgramNumber3	Coil Status	关
3	diProgramNumber4	Coil Status	关
4	diProgramNumber5	Coil Status	关
5	diProgramNumber6	Coil Status	关
6	diProgramNumber7	Coil Status	关
7	diProgramNumber8	Coil Status	关
8	M_1_9	Coil Status	关
9	M_1_10	Coil Status	关

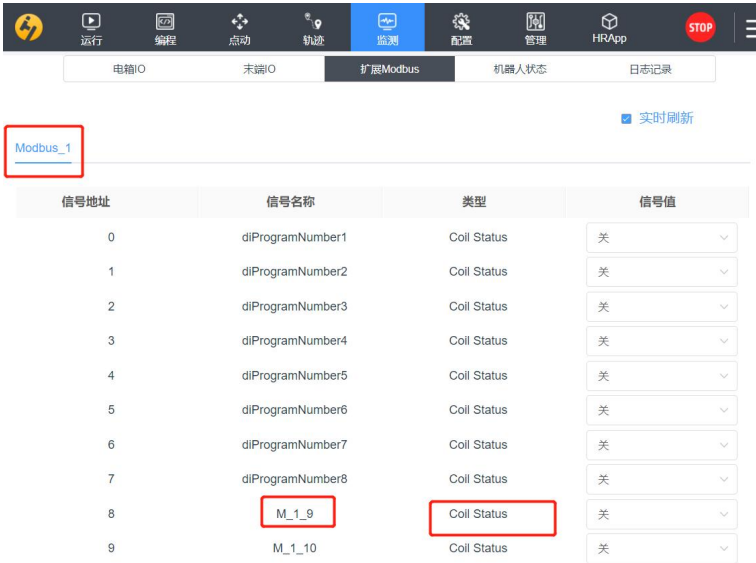
2.getModbus(读modbus地址位)

setModbus(写modbus地址位)

deviceName = 'Modbus_1' #modbus设备号

varName = 'M_1_9' #地址名称

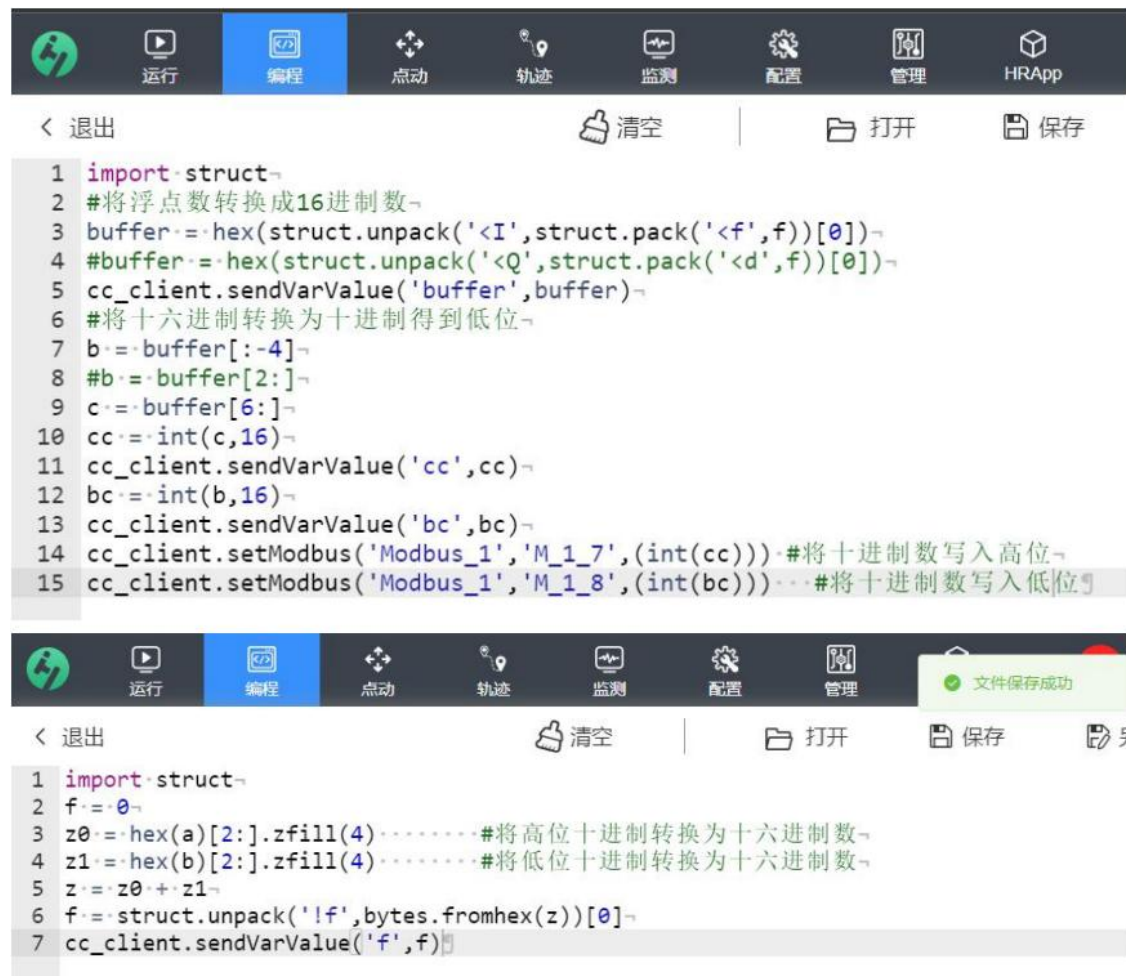
read_mb = cc_client.getModbus(deviceName,varName)



The screenshot shows the HANS ROBOT software interface. At the top, there is a navigation bar with icons for Run, Program, Jog, Trajectory, Monitor (selected), Parameters, Management, and HIRApp. Below the navigation bar, there are tabs for Electrical IO, Terminal IO, Expanded Modbus (selected), Robot Status, and Log Records. A 'Real-time Refresh' button is visible on the right. The main content area displays a table for the 'Modbus_1' device. The table has four columns: Signal Address, Signal Name, Type, and Signal Value. The signal 'M_1_9' is highlighted with a red box, and its type 'Coil Status' is also highlighted with a red box. The signal value is '关' (Off).

信号地址	信号名称	类型	信号值
0	diProgramNumber1	Coil Status	关
1	diProgramNumber2	Coil Status	关
2	diProgramNumber3	Coil Status	关
3	diProgramNumber4	Coil Status	关
4	diProgramNumber5	Coil Status	关
5	diProgramNumber6	Coil Status	关
6	diProgramNumber7	Coil Status	关
7	diProgramNumber8	Coil Status	关
8	M_1_9	Coil Status	关
9	M_1_10	Coil Status	关

特殊情况：1.读写浮点数



The screenshot displays the HANS ROBOT software interface, which includes a top toolbar with icons for Run, Program, Jog, Trajectory, Monitor, Configure, Manage, and HRApp. Below the toolbar is a menu bar with options like Exit, Clear, Open, and Save. The main area shows two code snippets for handling floating-point numbers.

Top Code Snippet (Reading Float):

```
1 import struct
2 #将浮点数转换成16进制数
3 buffer=hex(struct.unpack('<I',struct.pack('<f',f))[0])
4 #buffer=hex(struct.unpack('<Q',struct.pack('<d',f))[0])
5 cc_client.sendVarValue('buffer',buffer)
6 #将十六进制转换为十进制得到低位
7 b=buffer[:-4]
8 #b=buffer[2:]
9 c=buffer[6:]
10 cc=int(c,16)
11 cc_client.sendVarValue('cc',cc)
12 bc=int(b,16)
13 cc_client.sendVarValue('bc',bc)
14 cc_client.setModbus('Modbus_1','M_1_7',(int(cc)))#将十进制数写入高位
15 cc_client.setModbus('Modbus_1','M_1_8',(int(bc)))#将十进制数写入低位
```

Bottom Code Snippet (Writing Float):

```
1 import struct
2 f=0
3 z0=hex(a)[2:].zfill(4).....#将高位十进制转换为十六进制数
4 z1=hex(b)[2:].zfill(4).....#将低位十进制转换为十六进制数
5 z=z0+z1
6 f=struct.unpack('!f',bytes.fromhex(z))[0]
7 cc_client.sendVarValue('f',f)
```

A green notification bubble in the bottom right corner of the interface indicates "文件保存成功" (File saved successfully).

特殊情况：1.读字符串

```
import binascii
abc = cc_client.MBSlave_ReadHoldingRegisters(40,32)
while True:
    if int(abc[0]) == 0:
        abc = cc_client.MBSlave_ReadHoldingRegisters(40,32)
    if int(abc[0]) != 0:
        break
if int(abc[0]) != 0:
    k = 0
    L = len(abc)
    while True:
        if int(abc[k])!=0 and k<=L-2:
            k += 1
        else:
            break
    abc = abc[:k]
    i=0
    camera_2 = ''
    while True:
        ab = int(abc[i])
        camera_1 = binascii.unhexlify(hex(ab)[2:]).decode()
        camera_2 = str(camera_2)+str(camera_1)
        if i==k-1:
            break
        i=i+1
```

3.socket接口(tcp/ip通信 机器人做客户端)

ip = '127.0.0.1' #IP地址

port = '10003' #端口号

name = 'test' #通信号

cc_client.socket_open(ip,port,name) #打开socket通信

cc_client.socket_send_string('test' ,name) #输出字符串test

cc_client.socket_read_string(name) #接收输入数据

cc_client.socket_close(name) #关闭socket通信

谢谢